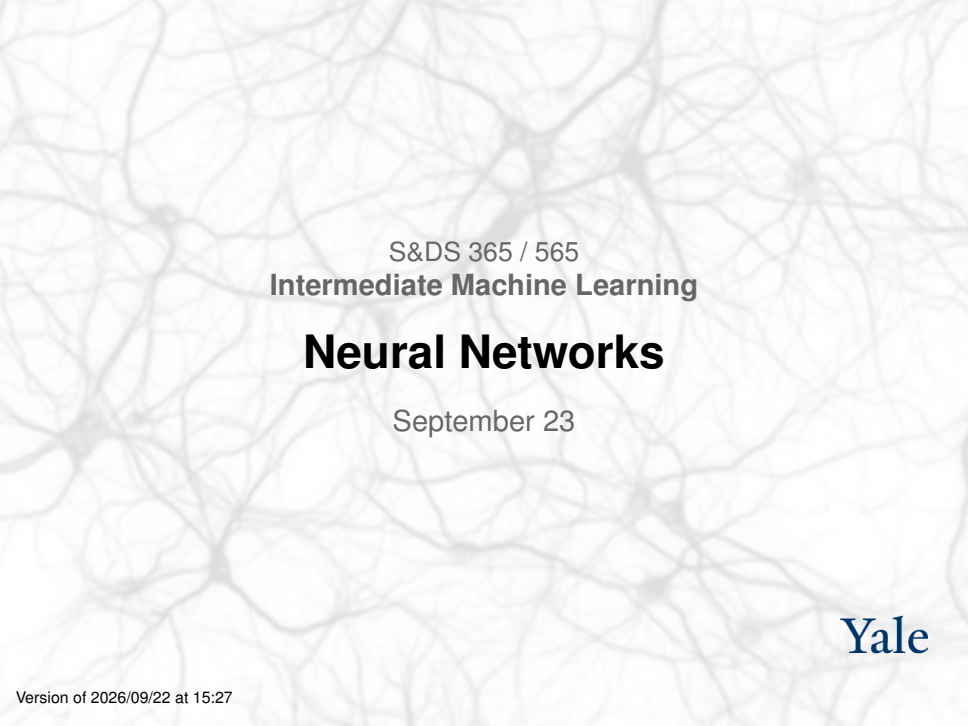




S&DS 365 / 565
Intermediate Machine Learning

Neural Networks

September 23

Yale

Reminders

- Assignment 1 out; due October 1 (week from this Thu)
- Quiz 2 end of class today
- Guest lecturer next Monday: Gaussian processes

Today: Neural nets

- ① Basic architecture of feedforward neural nets
- ② Examples (on board) and TensorFlow Playground
- ③ Next time: NTK and double descent

Kernels are central in ML

- Mercer kernels play a central role in machine learning
 - ▶ Can define similarity functions that are kernels for all kinds of data — graphs, molecules, text documents
 - ▶ Gaussian processes
 - ▶ Modern understanding of deep neural networks

Summary for kernels (so far)

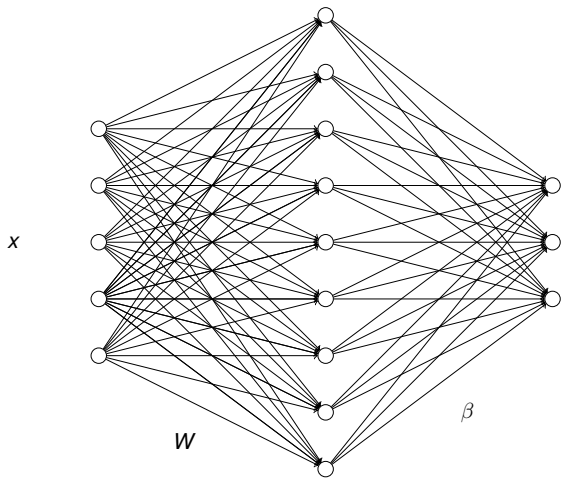
- Smoothing methods compute local averages, weighting points by a kernel. The details of the kernel don't matter much
- Mercer kernels using penalization rather than smoothing
- Defining property: Matrix $\mathbb{K}$ is always positive semidefinite
- Equivalent to a type of ridge regression in function space
- The curse of dimensionality limits use of both approaches

Neural net basics

Recall :-)

What does “Intermediate” imply?

- A second course in machine learning
- Assume familiar with things like PCA, bias/variance, maximum likelihood, **basics of neural nets**
- Have experimented with basic ML methods on some data sets
- Previous exposure to Python
- More on this later...



Interpretation

The parameter matrix W and vector β defines a (parametric) classification model, equivalent to a logistic regression:

$$\mathbb{P}(y | x) = \text{Softmax}(\beta^T \varphi(Wx))$$

Interpretation

The parameter matrix W and vector β defines a (parametric) classification model, equivalent to a logistic regression:

$$\mathbb{P}(y | x) = \text{Softmax}(\beta^T \varphi(Wx))$$

More generally, with intercept parameters:

$$\mathbb{P}(y | x) = \text{Softmax}(\beta^T \varphi(Wx + b) + \beta_0)$$

Softmax

If $v = (v_1, v_2, \dots, v_d)$ then

$$\text{Softmax}(v) = \frac{1}{\sum_j \exp v_j} (\exp v_1, \exp v_2, \dots, \exp v_d)$$

So, $\text{Softmax}(v)$ has components that are non-negative and sum to 1 (probabilities).

Nonlinearities

Add nonlinearity

$$h(x) = \varphi(Wx + b)$$

applied component-wise.

For regression, the last layer is just linear:

$$f(x) = \beta^T h(x) + \beta_0$$

For classification, we use Softmax .

Nonlinearities

Commonly used nonlinearities:

$$\varphi(u) = \tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$

$$\varphi(u) = \text{sigmoid}(u) = \frac{e^u}{1 + e^u}$$

$$\varphi(u) = \text{relu}(u) = \max(u, 0)$$

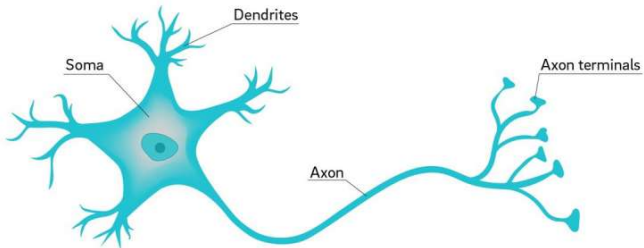
Without the activation function φ , we just have a regular linear model.

Nonlinearities

So, a neural network is nothing more than a parametric regression model with a restricted type of nonlinearity

Biological Analogy

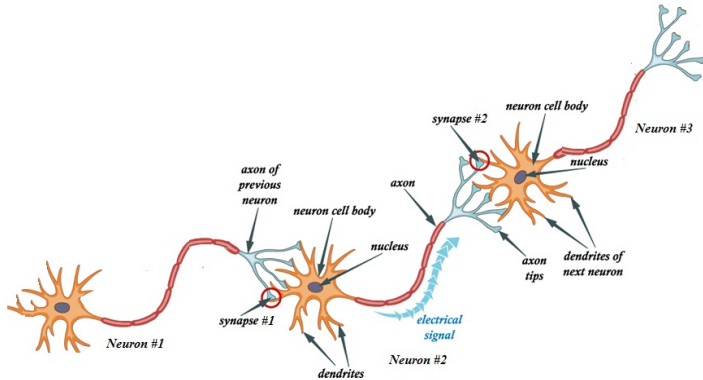
Neuron



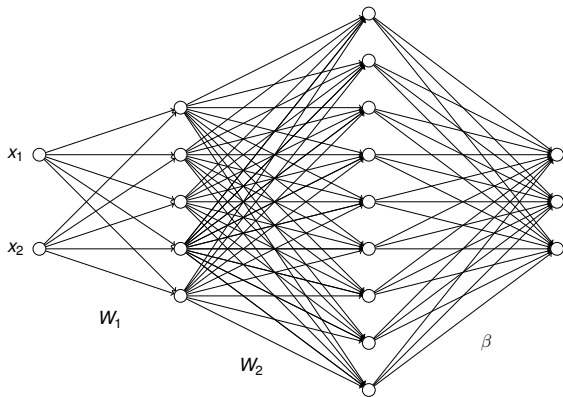
Biological Analogy

- The dendrites play the role of inputs, collecting signals from other neurons and transmitting them to the soma, which is the “central processing unit.”
- If the total input arriving at the soma reaches a threshold, an output is generated.
- The axon is the output device, which transmits the output signal to the dendrites of other neurons.

Biological Analogy



Two-layer perceptron



Corresponding model

This represents the family of models

$$\mathbb{P}(y | x) = \text{Softmax}(\beta^T \varphi(W_2 \varphi(W_1 x)))$$

Corresponding model

This represents the family of models

$$\mathbb{P}(y | x) = \text{Softmax}(\beta^T \varphi(W_2 \varphi(W_1 x)))$$

More generally, with intercept parameters:

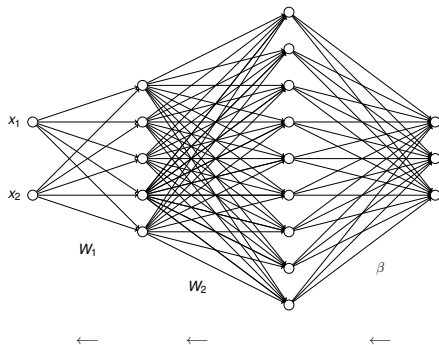
$$\mathbb{P}(y | x) = \text{Softmax}(\beta^T \varphi(W_2 \varphi(W_1 x + b_1) + b_2) + \beta_0)$$

3: Backprop

Training

- The parameters are trained by stochastic gradient descent.
- To calculate derivatives we just use the chain rule, working our way backwards from the last layer to the first.

High level idea



Start at last layer, send error information back to previous layers

4: Examples and demos

Example

We have a neural network given by $f(x) = \beta^T \varphi(Wx)$ where $x \in \mathbb{R}^3$, $\varphi(u) = \max(u, 0)$ is the ReLU activation function, and

$$W = \begin{pmatrix} 0 & -1 & 2 \\ 2 & 1 & -4 \end{pmatrix}$$

$$\beta^T = (1, 2)$$

- 1 What does the network look like?
- 2 What is the value of $f(x)$ if $x^T = (1, 1, 1)$?
- 3 What is the value of $f(x)$ if $x^T = (-1, 1, 0)$?

Interactive examples

`https://playground.tensorflow.org/`

What's going on?

- These models are curiously robust to overfitting
- Why is this?
- Some insight: Kernels and double descent

Next week!

Summary

- Neural nets are layered linear models with nonlinearities added
- Trained using stochastic gradient descent with backprop
- Can be automated to train complex networks (with no math!)